

CHAPITRE 4

GESTION DE LA MÉMOIRE

GESTION DE MÉMOIRE PAR LISTE CHAINÉE, PAR TABLE DE BITS

Année 2022/2023

INTRODUCTION

- La mémoire physique sur un système se divise en deux catégories :
 - la mémoire vive : composée de circuit intégrés, donc très rapide.
 - la mémoire de masse (secondaire) : composée de supports magnétiques (disque dur, bandes magnétiques...), qui est beaucoup plus lente que la mémoire vive..

INTRODUCTION

- La mémoire est une ressource rare. Il n'en existe qu'un seul exemplaire et tous les processus actifs doivent la partager.
- Si les mécanismes de gestion de ce partage sont peu efficaces, l'ordinateur sera peu performant, quelque soit la puissance de son processeur.
- ➔ **Comment gérer la mémoire pour le stockage des processus concurrents ??**

ALLOCATION CONTIGÜE

- L'allocation contigüe consiste à trouver dans la RAM un ensemble d'octet (partition) contigüe supérieur ou égale à la taille du processus à allouer.
- Deux stratégies ont été utilisées :
 - 1- partition taille fixe
 - 2- partition taille variable

PARTITION À TAILLE FIXE

- La mémoire est divisée en partition de taille fixe pas nécessairement égale dès le démarrage du système.
- Un processus ne peut être alloué que dans une seule partition → Une partition ne peut contenir qu'un seul processus.
- L'espace perdu d'une partition est appelé **fragmentation interne**.
- Chaque nouveau processus est placé dans une partition vide → problème de fragmentation interne due au fait que si la taille de partition est supérieure à l'espace recueilli par un certain processus, le reste de cette partition restera vide ce qui cause une perte d'espace mémoire.

PARTITION À TAILLE FIXE INÉGALES

- la mémoire est subdivisée en partitions de tailles inégales. Chaque nouveau processus est placé dans la file d'attente de la plus petite partition qui peut le contenir. Cette façon de faire peut conduire à faire attendre un processus dans une file, alors qu'une autre partition pouvant le contenir est libre.
- Il existe deux méthodes de gestion :
 - **File d'attente pour chaque partition**
 - **File d'attente unique**

PARTITION À TAILLE FIXE INÉGALES

- Le sous système chargé de la gestion de mémoire met en place une structure des données appelée table des partitions ayant pour rôle l'indication de l'état (libre ou occupée) de chaque partition en identifiant chacune soit par son adresse soit par son numéro.

PARTITION À TAILLE FIXE INÉGALES

- **File d'attente pour chaque partition** : il s'agit d'associer à chaque partition une file d'attente. Chaque processus est placé dans la file de la petite partition qui peut la contenir.

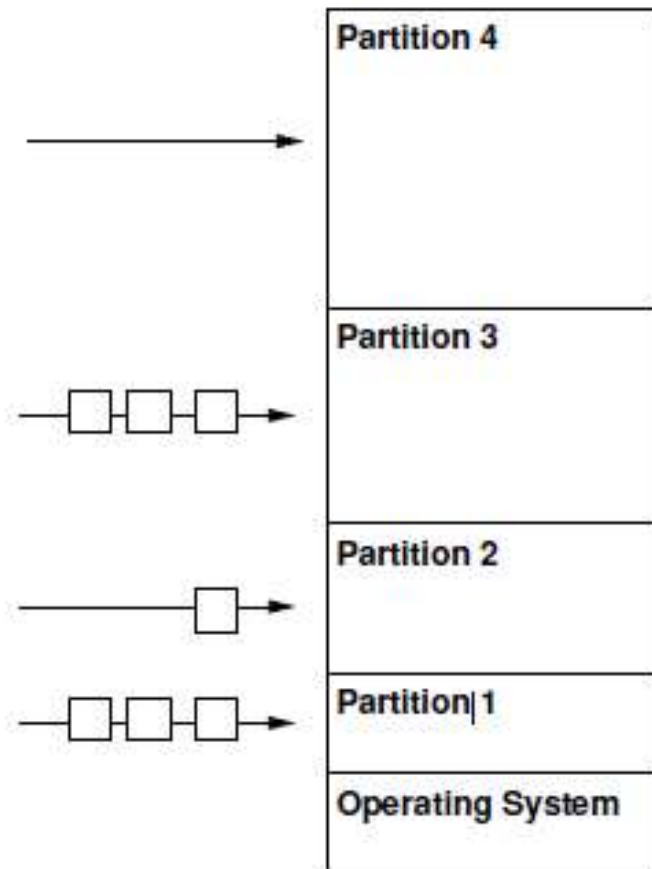
Inconvénients :

- ☐ on perd en général de la place au sein de chaque partition
- ☐ il peut y avoir des partitions inutilisées (leur file d'attente est vide)

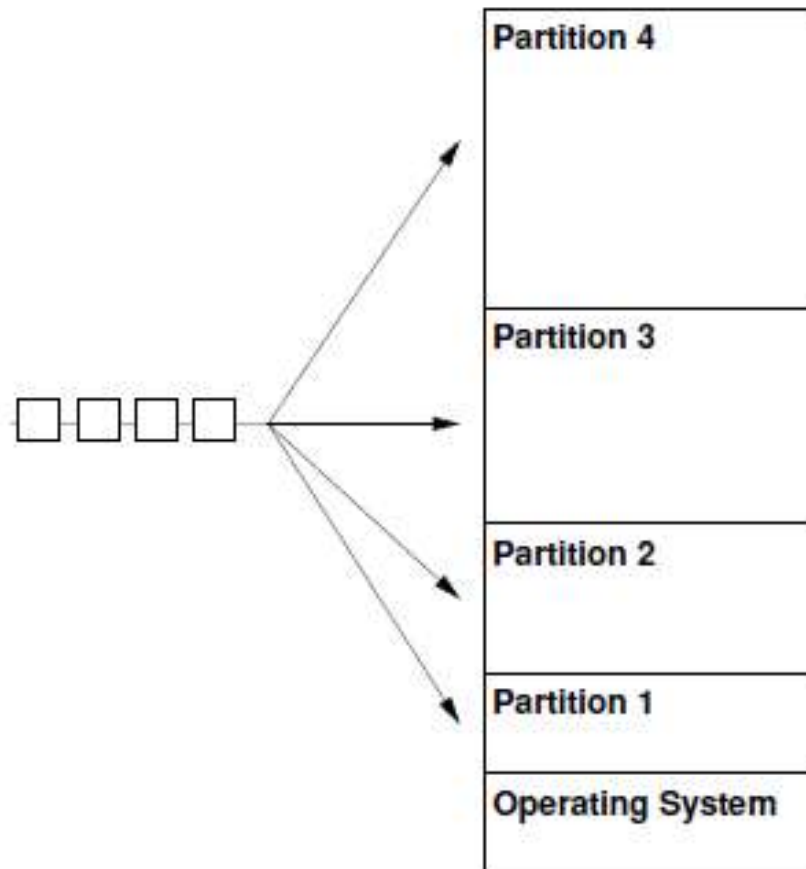
PARTITION À TAILLE FIXE INÉGALES

- **File d'attente unique** : on associe une file d'attente à toutes les partitions. Dès qu'une partition se libère, on identifie les processus qui peuvent être y alloué. Il existe deux stratégies :
 - Dès qu'une partition se libère, on lui affecte la première tâche de la file qui peut y tenir. Inconvénient : on peut ainsi affecter une partition de grande taille à une petite tâche et perdre beaucoup de place
 - Dès qu'une partition se libère, on lui affecte la plus grande tâche de la file qui peut y tenir. Inconvénient : on pénalise les processus de petite taille. L'alternative à cette approche consiste à n'utiliser qu'une seule file d'attente : dès qu'une partition se libère, le système y place le premier processus de la file qui peut y tenir. Cette solution réduit la fragmentation interne de la mémoire.

PARTITION À TAILLE FIXE INÉGALES



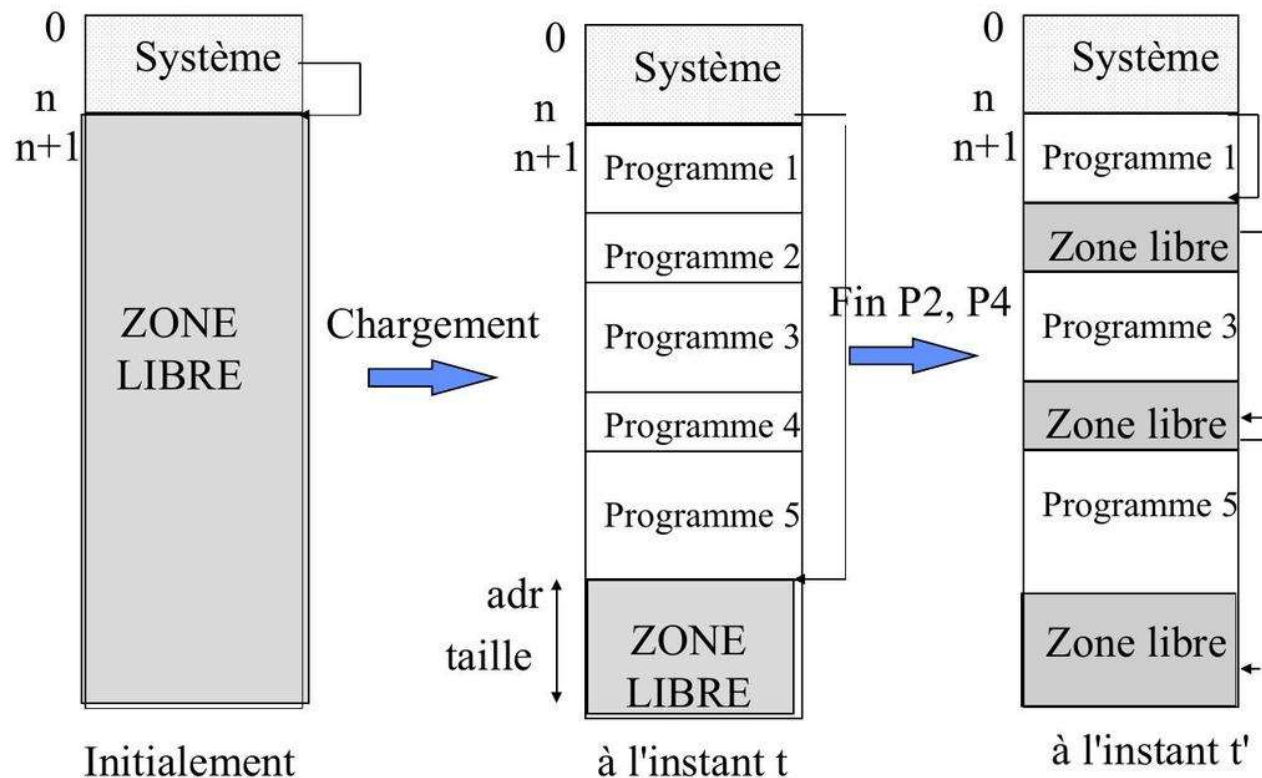
File d'attente par partition



File d'attente unique

PARTITION À TAILLE VARIABLE : SYSTÈME MULTIPROGRAMMÉ

- La mémoire centrale au démarrage du système est vide. Les partitions sont créées au fur et à mesure de l'arrivée et de départ des processus



PARTITION À TAILLE VARIABLE : SYSTÈME MULTIPROGRAMMÉ

- La mémoire est formée d'un ensemble de partition occupée.
- Allouer un programme consiste à lui trouver une partition de taille supérieure ou égale à sa taille.
- Des stratégies de placement des processus sont :
 - **First Fit (première zone libre)** : liste des trous par adresse. C'est la première partition libre qui peut contenir le processus.
 - **Best Fit (meilleur ajustement)** : liste des trous par taille croissante. C'est la plus petite partition qui peut contenir le processus.
 - **Worst Fit (plus grand résidu)** : liste des trous par taille décroissante. La plus grande partition qui peut contenir un processus.
- Pour un fonctionnement correct, il faut en moyenne, que le nombre de zones libres soit la moitié du nombre de zones allouées.

LE VA ET VIENT : LE SWAPPING

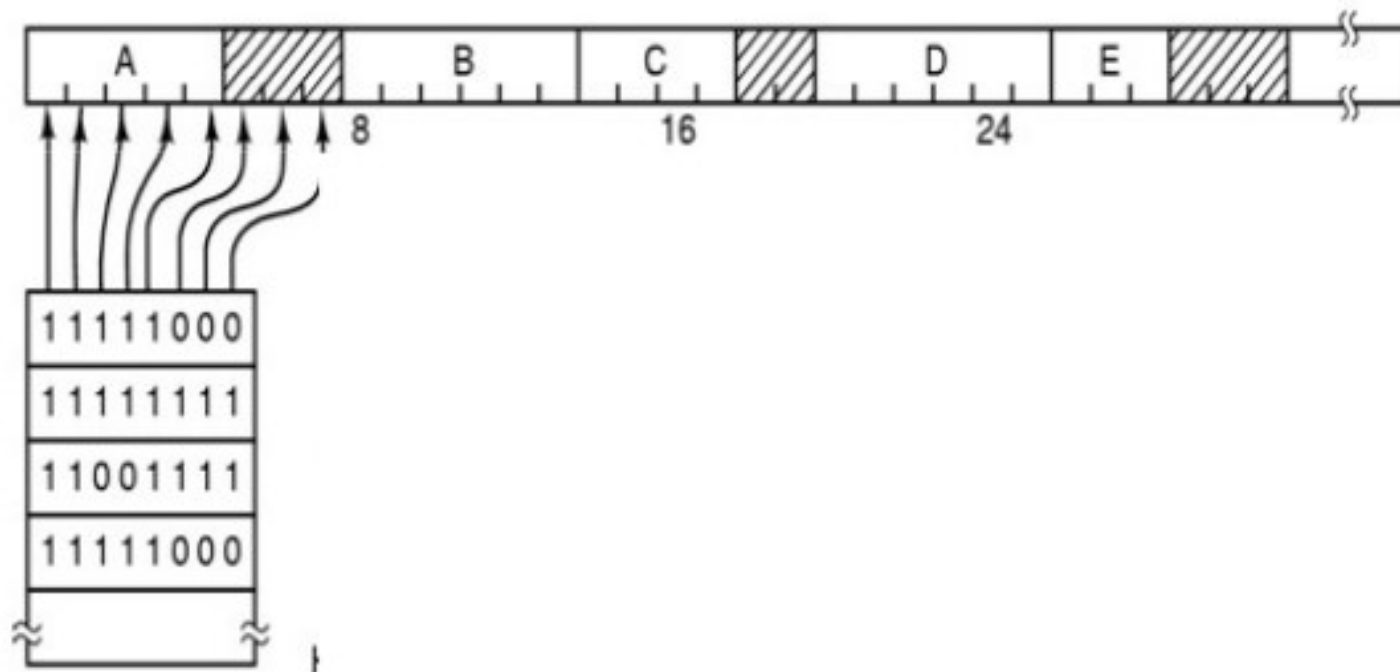
- La mémoire est une ressource qui ne peut pas contenir tous les processus actifs.
- Il est alors nécessaire de déplacer quelques processus inactifs (soit bloqués, soit prêts) dans le disque dure (**zone de swap**) et de les ramener en mémoire avant de l'exécuter
➔ ce moment de va et vient (**swapping**) est assuré par un programme : **le swappeur**.
- Il est à noter qu'il existe deux types de stratégies :
 - soit on alloue une place fixe dans le swap, pour chaque processus créé,
 - soit on utilise le swap comme une grande zone de stockage dans laquelle les processus sont écrits selon les besoins et à un endroit déterminé en fonction de l'occupation au moment de l'écriture.

GESTION DE LA MÉMOIRE

- Dans un système d'allocation contigüe partition variable, la mémoire est divisée en unités d'allocation qui peuvent être des zones libres et des zones occupées.
- L'unité d'allocation est 5k.
- La OS doit avoir une trace de l'état de la RAM.
- Deux stratégies ont été utilisées :
 - table de bits
 - liste chaînée

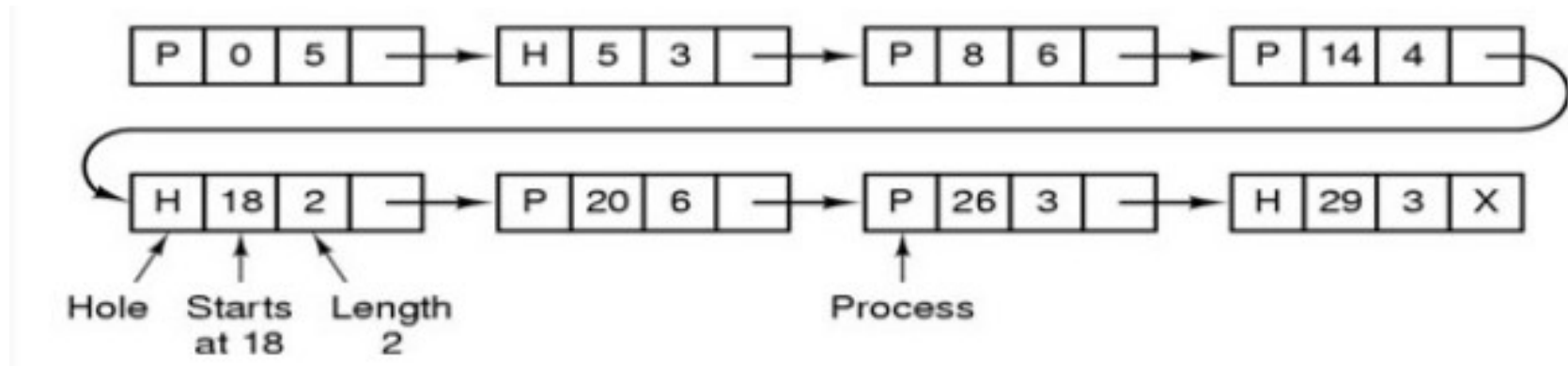
GESTION DE LA MÉMOIRE PAR TABLE DE BITS

- **Table de bits** : on associe à la RAM une table de bit décrivant par chaque case une unité d'allocation : 1 si l'unité est occupée, 0 si l'unité est libre

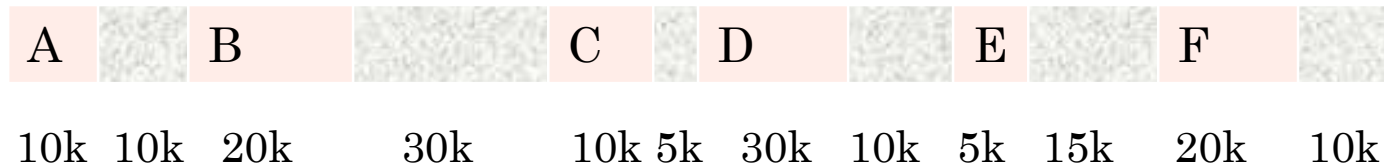


GESTION DE LA MÉMOIRE PAR LISTE CHÂÎNÉE

- **Liste chaînée** : on associe à la RAM une liste chaînée dont les éléments contiennent 4 champs
 - P (processus) ou T (trou, hole)
 - Adresse de départ
 - Taille **en nombre d'unités d'allocation**
 - Un pointeur vers l'élément suivant de la liste



EXERCICES



Représenter l'état de la mémoire dans une allocation contigüe taille variable et une stratégie de placement first fit. Soit les évènements suivants:

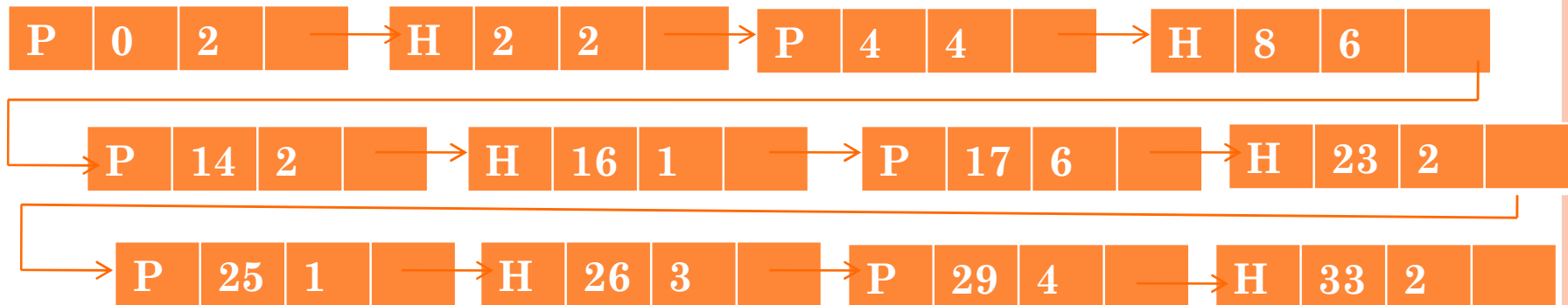
- Arrivé du processus G 20k
- Départ du processus B
- Arrivé du processus H 15k
- Départ du processus E
- Arrivé du processus I 40k

SOLUTION

Table de bit au début

A		B		C		D		E		F	
10k	10k	20k	30k	10k	5k	30k	10k	5k	15k	20k	10k
1	1	0	0	1	1	1	1	1	0		
0	0	0	0	0	1	1	0	0	1		
1	1	1	1	1	0	0	1	1	0		
0	0	1	1	1	1	0	0	0			

Liste chaînée au début

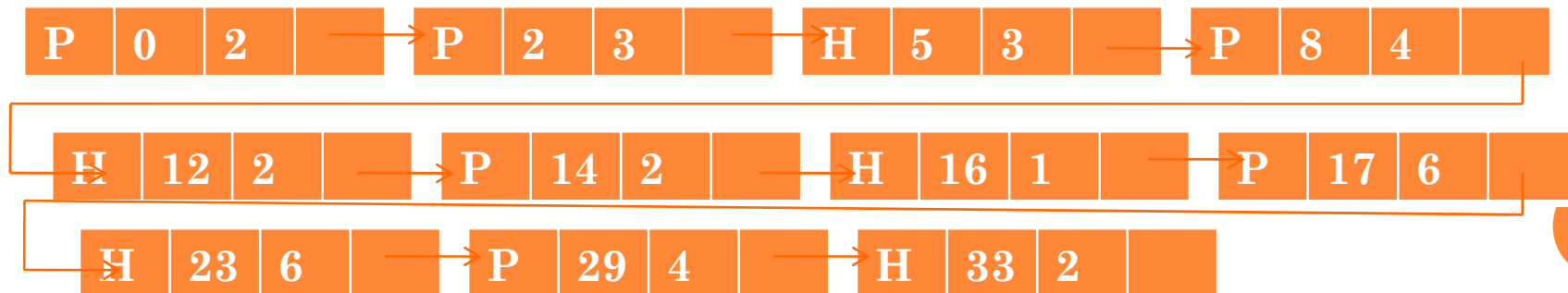


SOLUTION

Table de bit après

A	H		G		C		D			F	
10k	15k	15k	20k	10K	10k	5k	30k		30k	20k	10k

1	1	1	1	1	0	0	0	1
1	1	1	0	0	1	1	0	1
1	1	1	1	1	0	0	0	0
0	0	1	1	1	1	0	0	



ALLOCATION NON CONTIGÛE

- Il s'agit de pouvoir allouer les processus dans la RAM dans des adresses non consécutives → c'est pouvoir diviser les processus

Mémoire virtuelle

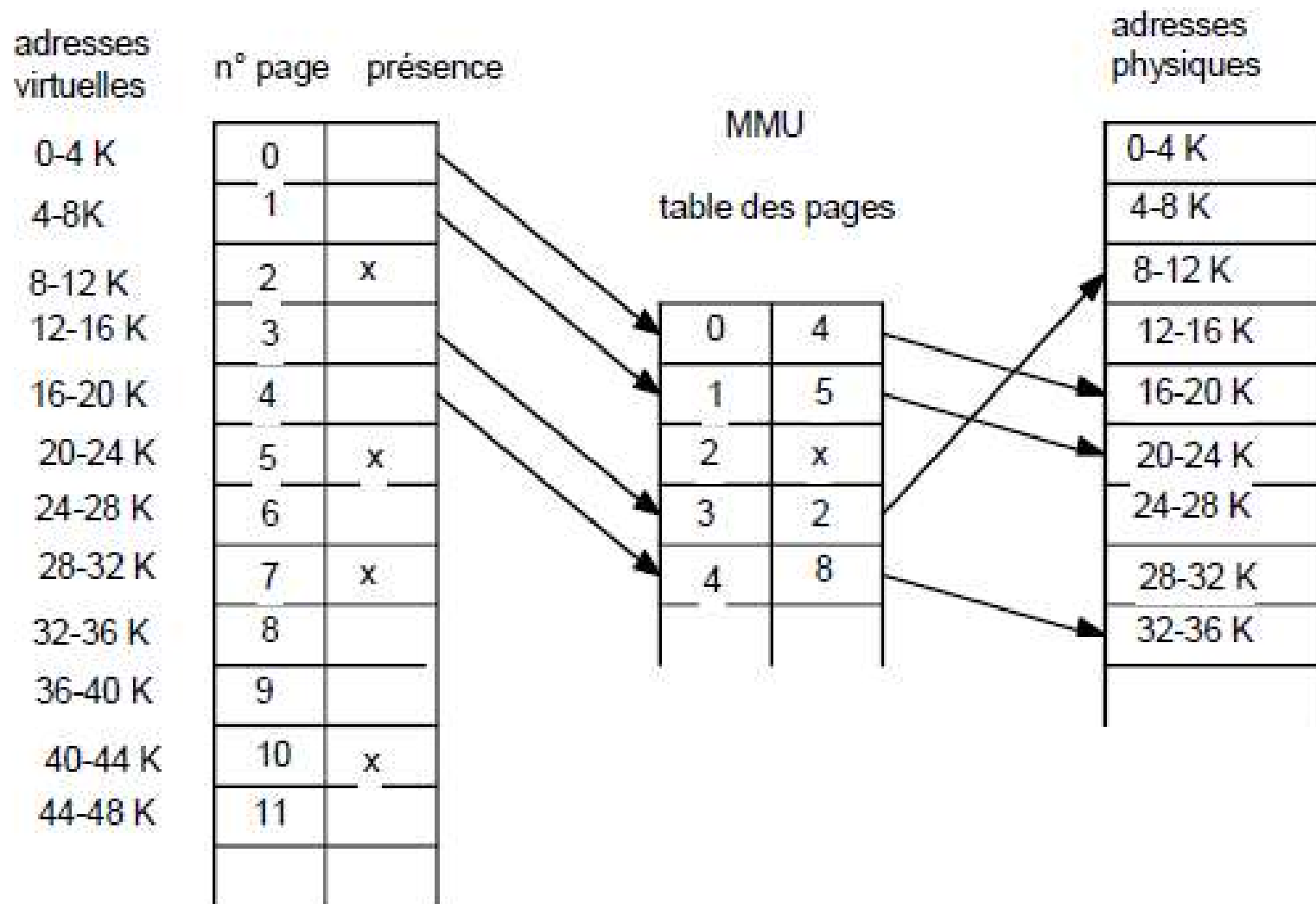
- Elle permet de fournir un espace d'adressage indépendant de celui de la mémoire physique
→ Possibilité d'exécuter des programmes dont la taille dépasse la mémoire disponible : le système garde en mémoire des parties utilisées du programme et stocker le reste dans la zone du swap
- physiquement la mémoire virtuelle est composée de RAM + zone de swap.
- Les instructions d'un programme sont référencées en utilisant **des adresses virtuelles** qui doivent être converties en adresses physiques grâce à une unité appelée **MMU**:
(Memory Management Unit)

ALLOCATION NON CONTIGÜE

Pagination

- L'espace d'adressage d'un processus est divisé en petites unités de taille fixe appelées **pages**.
- La MC est elle aussi découpée en unités physiques de même taille appelées **cadres**.
- Les échanges entre MC et disques ne portent que sur des pages entières. De ce fait, l'espace d'adressage d'un processus est potentiellement illimité (limité à l'espace mémoire total de la machine). On parle alors d'**adressage virtuel**.
- On associe à chaque processus **une table de pages** qui décrit l'état des pages (bit de présence, bit de modification, case associée)
- On dira qu'une page est **mappée** ou **chargée** si elle est physiquement présente en mémoire.

EXEMPLE DE PAGINATION



EXEMPLE DE PAGINATION

- Dans l'exemple précédent, les pages ont une taille de 4 Ko. L'adresse virtuelle 12292 correspond à un déplacement de 4 octets dans la page virtuelle 3 (car $12292 = 12288 + 4$ (et $12288 = 12 * 1024$)).
- La page virtuelle 3 correspond à la page physique 2. L'adresse physique correspond donc à un déplacement de 4 octets dans la page physique 2, soit : $(8 * 1024) + 4 = 8196$
- Par contre, la page virtuelle 2 n'est pas mappée. Une adresse virtuelle comprise entre 8192 et 12287 donnera lieu à **un défaut de page. Il y a défaut de page quand il y a un accès à une adresse virtuelle** correspondant à une page non mappée. En cas de défaut de page, un déroutement se produit (trap) et le processeur est rendu au SE.

PAGINATION

Le système doit alors effectuer les opérations suivantes :

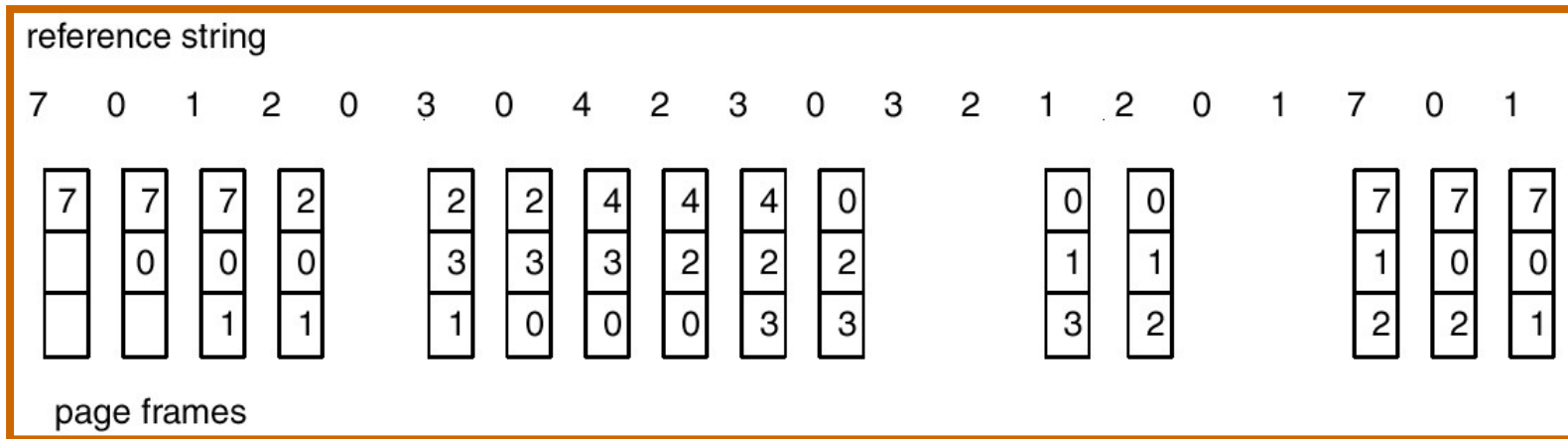
- déterminer la page à charger (page victime)
- déterminer la page à décharger sur le disque pour libérer un cadre
- lire sur le disque la page à charger
- modifier la table de bits et la table de pages

La sélection de page est réalisée par les **algorithmes de remplacement**

ALGORITHMES DE REMPLACEMENT

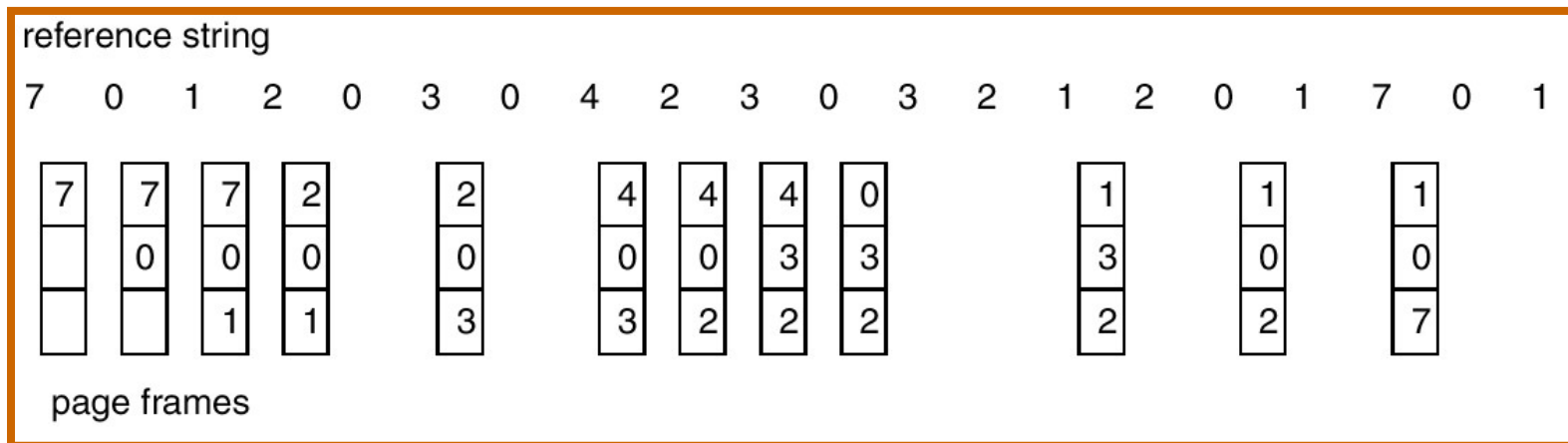
- **FIFO (first in, first out)** : la page la plus ancienne dans la RAM
- **LRU (least recently used)** : il s'agit de remplacer avec la page qui est restée inutilisée depuis le plus longtemps
- **Algorithme optimal** : il s'agit de remplacer avec la page qui sera utilisée dans le futur le plus lointain

REEMPLACEMENT FIFO



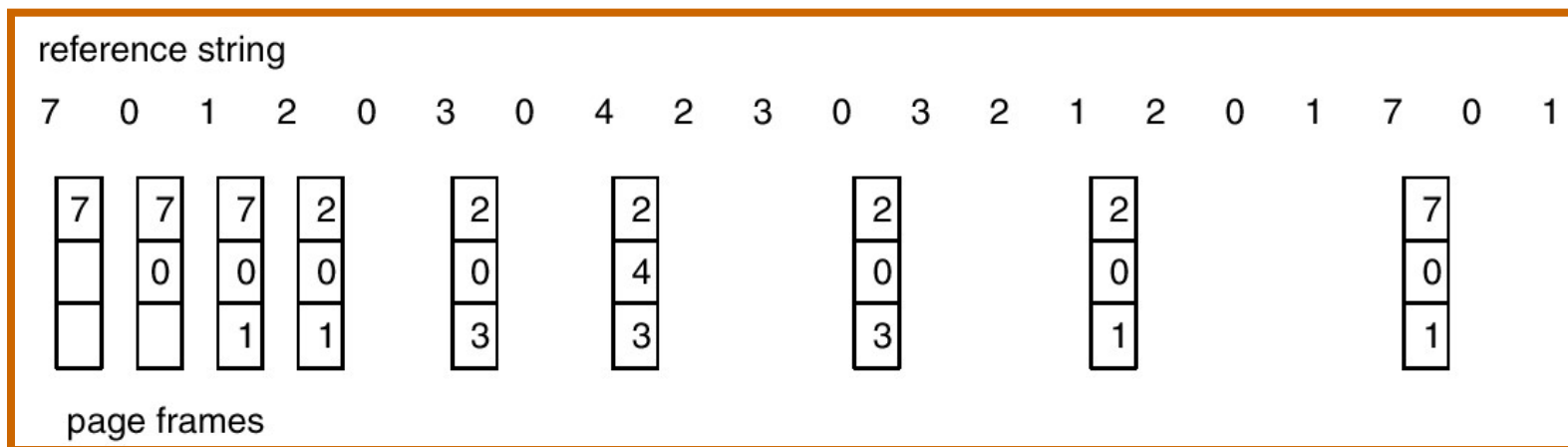
la page la plus ancienne

REEMPLACEMENT LRU



la page inutilisée depuis le plus
longtemps

REEMPLACEMENT OPTIMAL



la page utilisée dans le futur le plus
lointain

LA SEGMENTATION

- l'espace d'adressage d'un processus est divisé en **segments**, générés à la compilation.
- Chaque segment est repéré par son numéro S et sa longueur variable L .
- Un segment est un ensemble d'adresses virtuelles contiguës.
- Contrairement à la pagination, la segmentation est "connue" du processus : une adresse n'est plus donnée de façon absolue par rapport au début de l'adressage virtuel; une adresse est donnée par un couple (S, d) , où S est le n° du segment et d le déplacement dans le segment, d appartient à $[0, L[$.

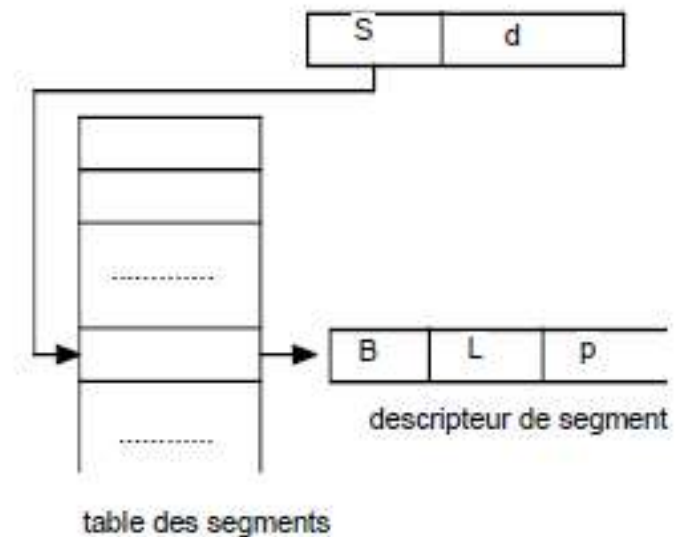
LA SEGMENTATION

- Pour calculer l'adresse physique, on utilise une **table des segments** :

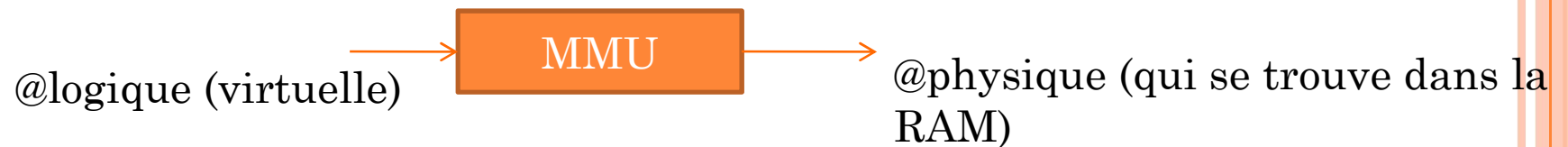
B : adresse de base (adresse physique de début du segment)

L : longueur du segment ou limite

p : protection du segment



- L'adresse physique correspondant à l'adresse virtuelle (S , d) sera donc $B + d$, si $d \leq L$
- La segmentation simplifie la gestion des objets communs (rangés chacun dans un segment), notamment si leur taille évolue dynamiquement.



Exemple :

Processeur 16 bits

➔ Mémoire virtuelle $2^{16} = 2^6 \text{ ko} = 64 \text{ ko}$

Mémoire physique = 32 ko (RAM)

Taille de page = 4 ko

Nombre de pages virtuelles = $64/4 = 16$

Nombre de cases = $32/4 = 8$ cases